Mikrodatorteknik (TMIK13) – Tenta A med svar

Fil: GetInsperaExamPdf.pdf · 13 uppgifter · 50 p

> **Hur du läser:** Varje uppgift har först **Svar**, sedan en kort *Förklaring* på enkel svenska. Kontrollera gärna mot dina egna anteckningar – och skriv gärna egna kommentarer i filen.

1. NVIC (1 p)

Vad står förkortningen NVIC för?

Svar: NVIC = **Nested Vectored Interrupt Controller**.

Förklaring: Det är den del i ARM Cortex-M som hanterar alla avbrott (interrupts). "Nested" = ett avbrott med högre prioritet kan avbryta ett avbrott med lägre prioritet. "Vectored" = varje avbrott har en egen adress i en tabell (vektortabellen), så CPU:n hoppar direkt till rätt ISR.

2. ADC – potentiometer (1 p)

ADC ger 0–4095 för 0–3.3 V. Digitalt värde 2854 – vilken spänning?

Svar: ≈ 2.30 V

Förklaring: U = 2854 / 4095 × 3.3 = 2.2999 ≈ **2.30 V**. ADC:n delar 3.3 V i 4096 lika steg, så man räknar bara "hur stor andel av max" värdet är.

Grundformel ADC: **U = värde / 4095 × 3.3 V** (vissa lärare använder 4096 – ger nästan samma svar).

3. Big / little endian (4 p)

R2 = 0xAA4512B7 skrivs till adress 0x20002218.

Adress	Big endian	Little endian
0x20002218	**0xAA**	**0xB7**
0x20002219	**0x45**	**0x12**
0x2000221A	**0x12**	**0x45**
0x2000221B	**0xB7**	**0xAA**

Förklaring: Dela talet i byte: **AA 45 12 B7**.

- **Big endian** = "big end first": den **mest** signifikanta byten (AA) på lägsta adressen – samma ordning som du läser talet.
- **Little endian** = "little end first": den **minst** signifikanta byten (B7) på lägsta adressen – baklänges.
- ARM Cortex-M (STM32) är **little endian**.

(OBS: tentan skriver adresserna som 0x2002218 osv – det är ett stavfel, det menas 0x20002218.)

4. Pull-up / pull-down (2 p)

Svar:

- **Pull-up:** ett motstånd mellan pinnen och **VCC (3.3 V)**. När inget annat driver pinnen läses den som **1 (hög)**.
- **Pull-down:** ett motstånd mellan pinnen och **GND**. När inget annat driver pinnen läses den som **0 (låg)**.

När används det? När en ingång annars skulle "flyta" (vara odefinierad), t.ex. en knapp som bara kopplar till GND (då behövs pull-up), eller en sensor med **open-drain/open-collector**-utgång som bara kan dra ner till GND. Även I2C-linjer behöver pull-up.

Hur aktiveras det? Antingen med ett yttre motstånd (typ 4.7–10 kΩ) eller med det **inbyggda** motståndet i STM32: registret **PUPDR** (i CubeMX: GPIO → "Pull-up" / "Pull-down").

Förklaring: En ingång som inte är kopplad till något är som en antenn – den kan läsas som 0 eller 1 slumpmässigt. Motståndet ger ett "standardläge" men är så stort att knappen/sensorn lätt kan ändra nivån.

5. Assembler (3 p)

```
my_function:
  MOVS R0, #63
0x8002784:
  NOP
  NOP
  NOP
  SUBS R0, R0, #1
  BNE 0x8002784
```

Fråga	Svar
R0 initialt	**63**
SUBS R0, R0, #1	**Subtraherar 1 från R0 och uppdaterar flaggor**
BNE beror på	**Zero**
Antal NOP	**189**
När R0 = 0	**Programmet fortsätter**
Syfte	**Åstadkomma en kort fördröjning**

Förklaring:

- **MOVS** lägger in talet 63 i R0.
- **S** i slutet av SUBS betyder "**S**et flags" – flaggorna (N, Z, C, V) uppdateras.
- **BNE** = Branch if Not Equal = hoppa om **Z-flaggan är 0** (dvs resultatet blev inte 0).
- Loopen körs 63 gånger (63, 62, … 1). Varje varv har **3 NOP** → 63 × 3 = **189**.
- När R0 blir 0 sätts Z = 1, BNE hoppar inte, och programmet går vidare till nästa rad.
- NOP gör ingenting – loopen bara "slösar tid" = en fördröjning (delay).

6. DMA (2 p)

Svar: DMA = **Direct Memory Access**.

En DMA-styrenhet kan flytta data **utan att CPU:n gör jobbet**: mellan kringenhet ↔ minne (t.ex. ADC → RAM, RAM → UART/SPI) eller minne ↔ minne. CPU:n ställer bara in källa, destination, antal och sedan sköter DMA:n överföringen. När den är klar kan den ge ett avbrott (t.ex. "transfer complete", även "half transfer").

Användningsområden:

- ADC som samplar många värden kontinuerligt → DMA lägger dem i en buffert i RAM.
- Skicka/ta emot mycket data på UART, SPI, I2C (t.ex. en display, SD-kort).
- Ljud (DAC/I2S) där data måste komma i jämn takt.
- Kopiera stora minnesblock.

Fördel: CPU:n blir avlastad – kan göra annat eller sova (spar ström), och överföringen går snabbt och i jämn takt.

Förklaring: Tänk dig DMA som en "flyttgubbe": CPU:n säger "flytta 100 värden från ADC till den här arrayen" och kan sedan göra annat.

7. RAM-minne (3 p)

Figuren: 44-pin TSOP II med A0–A17, I/O0–I/O15, /BHE, /BLE.

Fråga	Svar
Adresspinnar	**18** (A0–A17)
Ordlängd	**16 bitar** (I/O0–I/O15)
Antal byte	**524288**

Förklaring:

- Räkna **A-pinnarna**: A0 till A17 = 18 st.
- Räkna **data-pinnarna** (I/O): I/O0 till I/O15 = 16 st → varje adress lagrar 16 bitar = 2 byte.
- Antal adresser = 2^18 = 262 144. Varje adress har 2 byte → 262 144 × 2 = **524 288 byte** (= 512 kB).
- /BHE och /BLE (byte high/low enable) låter dig välja att bara läsa övre eller undre byten – de är inte adresspinnar.

8. PLL (2 p)

Svar: PLL = **Phase-Locked Loop** (faslåst slinga). Den används för att **multiplicera upp** en låg klockfrekvens till en högre systemklocka.

Hur den fungerar: En VCO (spänningsstyrd oscillator) genererar en hög frekvens. Den delas ner och jämförs i en **fasdetektor** med referensklockan. Skillnaden filtreras och styr VCO:n så att den låser sig på exakt rätt frekvens.

I STM32L433:

- Klockkälla: HSI16 (16 MHz), MSI eller HSE (extern kristall).
- Först delas den med **M**, sedan multipliceras med **N** (i VCO:n), sedan delas med **R** (systemklocka), **P** och **Q** (andra utgångar, t.ex. USB/SAI).
- f_ut = f_in / M × N / R. Exempel: 16 MHz / 1 × 10 / 2 = **80 MHz** (max för L433).

Förklaring: Kristaller och interna oscillatorer finns bara för låga frekvenser, men CPU:n vill gå snabbt. PLL:en “multiplicerar” klockan och ser till att den ändå är lika stabil som referensen.

9. GPIO-register (3 p)

Svar:

- **MODER** (mode): väljer läge för varje pinne – **Input, Output, Alternate function** (t.ex. UART, TIM, I2C) eller **Analog** (ADC/DAC). 2 bitar per pinne.
- **OTYPER** (output type): **push-pull** eller **open-drain** för utgången.
- **OSPEEDR** (output speed): hur snabbt utgången kan slå om (low/medium/high/very high). Högre = snabbare flanker men mer störningar/ström.
- **PUPDR** (pull-up/pull-down): slår på intern **pull-up**, **pull-down** eller ingen.
- **IDR** (input data register): **läsa** pinnens nivå (0/1).

- **ODR** (output data register): **skriva** vad utgången ska ge (0/1).

Minnesregel: MODE = vad är pinnen, TYPE = hur driver den, SPEED = hur snabbt, PUPD = motstånd, I = in, O = ut.

10. Sant/falskt om timers (3 p)

Svar (bocka i dessa – de är SANNA):

- ✔ Vissa timers kan räkna även när CPU sover, om deras klockkälla är aktiv. *(t.ex. LPTIM, RTC)*
- ✔ Auto-reload registret (ARR) bestämmer vid vilket värde timern återgår till 0.
- ✔ När räknaren slår om kan en interrupt genereras om den är aktiverad. *(update event)*

FALSKA (bocka inte):

- ✘ "En timer har alltid samma frekvens som CPU-kärnan" – nej, den kan ha annan klocka och delas med prescalern.
- ✘ "Prescalern används för att öka counterns räkningshastighet" – nej, prescalern **delar** klockan → räknaren går **långsammare**.
- ✘ "PWM kan generera en analog spänning direkt" – nej, PWM är bara 0/3.3 V-pulser. Man behöver ett lågpassfilter (RC) för att få en analog medelspänning.

Förklaring: Felbockning ger −1, så bocka bara det du är säker på.

11. PWM-styrning (5 p)

Givet: 80 MHz, PWM ≈ 5 kHz, 100 steg.

Register	Svar
PSC	**159**
ARR	**99**
CCR1 (25 %)	**25**
PSC för max frekvens	**0**
Frekvens	**800** kHz

Förklaring:

- 100 steg → räknaren räknar 0…99 → **ARR = 100 − 1 = 99**.
- Räknaren måste då gå med 5 kHz × 100 = 500 kHz. 80 MHz / 500 kHz = 160 → **PSC = 160 − 1 = 159**.
- Duty = CCR / (ARR+1) → 25 % av 100 = **25**.
- Max frekvens: dela inte alls → PSC = 0 → 80 MHz / 100 = **800 kHz** (långt över hörbart).

Formel att kunna: **f_PWM = f_clk / ((PSC+1)·(ARR+1))**. Kom ihåg "−1" på både PSC och ARR!

12. Timer-avbrott (1 p)

PSC = 799, 80 MHz, interrupt 100 gånger per sekund.

Svar: ARR = 999

Förklaring:

- Timerklocka = 80 MHz / (799 + 1) = **100 kHz** (räknaren tickar 100 000 ggr/s).
- 100 interrupts/s → varje interrupt ska ske efter 100 000 / 100 = 1000 tick → **ARR = 1000 − 1 = 999**.
- *OBS:* Frågan säger både “100 gånger per sekund” och “en sekunds intervall” (motsägelse). Om de menar **1 sekund**: ARR = 100 000 − 1 = **99999** – men det ryms inte i en 16-bitars timer (max 65535), bara i en 32-bitars (TIM2). Därför är **999** troligast rätt.

Formel: **f_interrupt = f_clk / ((PSC+1)·(ARR+1))**

13. Bilbana – Bengt Bengtsson (20 p)

a) Pinnkonfiguration i CubeMX (4 p)

Pinne	Konfiguration	Varför
PA9 / PA10	Aktivera **I2C1** → PA9 = I2C1_SCL, PA10 = I2C1_SDA (alternate function, open-drain). Pull-ups (externa eller interna).	Displayen pratar I2C.
PC2 / PC3	**GPIO_EXTI2 / GPIO_EXTI3**, trigger på **fallande flank**, **Pull-up**. Aktivera EXTI2 och EXTI3 i NVIC.	Sensorerna är open-drain och drar till GND när en bil passerar → behöver pull-up, och fallande flank = “bil här”.
PA4	**GPIO_Input** med **Pull-up**.	Knappen kopplar till GND (se figuren) → läses 1 normalt och 0 när den trycks.
PB2	**GPIO_Output**, push-pull, ingen pull, startnivå **Low**.	Aktiv summer – ljuder när pinnen är 3.3 V.

Tid: Jag använder `HAL_GetTick()` (SysTick, 1 ms) för tidmätning – räcker för att visa två decimaler. (Alternativ: en timer, t.ex. TIM6, med 1 kHz-interrupt.)

b) Program (16 p)

Antaganden (skriv alltid ner sådana på tentan!):

1. LCD-drivrutin finns: `lcd_init()`, `lcd_clear()`, `lcd_set_cursor(rad, kol)`, `lcd_print(text)`.
2. Bilarna står precis **efter** mållinjen vid start → första gången sensorn triggar har bilen kört varv 1.
3. En bil kan ge flera flanker när den passerar → passager närmare än 300 ms ignoreras (debounce).
4. Tiden räknas från att det **långa** pipet startar.

Idé (enkelt förklarat): Programmet har tre lägen (tillstånd): **IDLE** (väntar på knapp), **RACING** (mäter tid), **FINISHED**. Sensorerna ger avbrott → i avbrottet sparar vi tiden och räknar varv. Main-loopen sköter knappen, summern och displayen.

```c
#include "main.h"
#include "lcd.h"
#include <stdio.h>

#define NUM_LANES    2
#define NUM_LAPS     5
#define DEBOUNCE_MS  300

typedef enum { STATE_IDLE, STATE_RACING, STATE_FINISHED } race_state_t;

typedef struct {
  volatile uint32_t last_pass;  // tid (ms) vid senaste passage
  volatile uint32_t last_lap;   // senaste varvtid (ms)
  volatile uint32_t total;      // total tid när 5 varv är klara (ms)
  volatile uint8_t  laps;       // antal körda varv
  volatile uint8_t  done;       // 1 = klar med 5 varv
} lane_t;

static lane_t lanes[NUM_LANES];
static volatile race_state_t state = STATE_IDLE;
static volatile uint32_t race_start;
static volatile uint8_t  winner_found = 0;
static volatile uint8_t  winner_beep  = 0;  // ISR säger åt main att pipa

/* ---------- Summer ---------- */
static void buzzer(int on) {
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_2, on ? GPIO_PIN_SET : GPIO_PIN_RESET);
}

static void beep(uint32_t ms) {
  buzzer(1);
  HAL_Delay(ms);
  buzzer(0);
}

static void start_signal(void) {   // 3 korta + 1 långt pip
  for (int i = 0; i < 3; i++) {
    beep(200);
    HAL_Delay(800);
  }
  beep(1000);
}

/* ---------- Lopp ---------- */
```

```c
static void reset_race(void) {
  for (int i = 0; i < NUM_LANES; i++) {
    lanes[i].laps = 0;
    lanes[i].last_lap = 0;
    lanes[i].total = 0;
    lanes[i].done = 0;
  }
  winner_found = 0;
  winner_beep = 0;
}

/* Knapp: aktiv låg. Returnerar 1 bara när knappen precis trycks ner. */
static int button_pressed(void) {
  static GPIO_PinState prev = GPIO_PIN_SET;
  GPIO_PinState now = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_4);
  int pressed = (prev == GPIO_PIN_SET && now == GPIO_PIN_RESET);
  prev = now;
  return pressed;
}

/* ---------- Avbrott från sensorerna (PC2, PC3) ---------- */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
  int i;
  if (GPIO_Pin == GPIO_PIN_2)      i = 0;   // spår 1
  else if (GPIO_Pin == GPIO_PIN_3) i = 1;   // spår 2
  else return;

  if (state != STATE_RACING || lanes[i].done) return;

  uint32_t now = HAL_GetTick();
  if (now - lanes[i].last_pass < DEBOUNCE_MS) return;  // samma bil igen

  lanes[i].last_lap  = now - lanes[i].last_pass;
  lanes[i].last_pass = now;
  lanes[i].laps++;

  if (lanes[i].laps >= NUM_LAPS) {
    lanes[i].done  = 1;
    lanes[i].total = now - race_start;  // stoppa totaltiden
    if (!winner_found) {                // första bilen i mål = vinnare
      winner_found = 1;
      winner_beep  = 1;
    }
  }
}

/* ---------- Display ---------- */
static void show_lane(int i, uint32_t now) {
  char buf[24];
  uint32_t lap = lanes[i].last_lap;
  uint32_t tot;

  if (lanes[i].done)              tot = lanes[i].total;
  else if (state == STATE_RACING) tot = now - race_start;
  else                            tot = 0;

  // skriv som sekunder med 2 decimaler utan float-printf
  snprintf(buf, sizeof(buf), "Spar %d: %lu.%02lus / %lu.%02lus", i + 1,
           lap / 1000, (lap % 1000) / 10,
           tot / 1000, (tot % 1000) / 10);
  lcd_set_cursor(i, 0);
  lcd_print(buf);
}

/* ---------- main ---------- */
int main(void) {
  HAL_Init();
```

```
  SystemClock_Config();
  MX_GPIO_Init();
  MX_I2C1_Init();

  buzzer(0);
  lcd_init();
  lcd_clear();
  lcd_print("Tryck start!");
  reset_race();

  uint32_t last_lcd = 0;
  uint32_t buzzer_off_time = 0;

  while (1) {
    uint32_t now = HAL_GetTick();

    /* Start / omstart */
    if (button_pressed()) {
      state = STATE_IDLE;            // stoppa ev. gammalt lopp
      reset_race();
      lcd_clear();
      lcd_print("Klara, fardiga...");
      start_signal();                // blockerar ok – loppet har inte börjat
      race_start = HAL_GetTick();
      for (int i = 0; i < NUM_LANES; i++) lanes[i].last_pass = race_start;
      lcd_clear();
      state = STATE_RACING;
    }

    /* Vinnare -> summer 2 s (utan att blockera) */
    if (winner_beep) {
      winner_beep = 0;
      buzzer(1);
      buzzer_off_time = now + 2000;
    }
    if (buzzer_off_time != 0 && now >= buzzer_off_time) {
      buzzer(0);
      buzzer_off_time = 0;
    }

    /* Båda klara -> loppet slut */
    if (state == STATE_RACING && lanes[0].done && lanes[1].done) {
      state = STATE_FINISHED;
    }

    /* Uppdatera displayen 10 ggr/s */
    if (state != STATE_IDLE && now - last_lcd >= 100) {
      last_lcd = now;
      show_lane(0, now);
      show_lane(1, now);
    }
  }
}
```

Förklaring av viktiga delar:

- **`volatile`** – variabler som ändras i ett avbrott måste vara volatile, annars kan kompilatorn “tro” att de aldrig ändras i main.
- **Avbrott (EXTI) för sensorerna** – så vi aldrig missar en bil, även om main-loopen håller på att skriva på displayen.
- **Debounce** – sensorn kan ge flera pulser när en bil passerar. Vi ignorerar passager inom 300 ms.

- **Varvtid** = nu − förra passagen. **Totaltid** = nu − starttid, och den “fryses” när bilen kört 5 varv.
- **Vinnarsummern** blockerar inte (ingen `HAL_Delay`) – vi sätter på den och stänger av den i main-loopen efter 2 s, så den andra bilens tid fortfarande mäts.
- **`%lu.%02lu`** – vi skriver sekunder och hundradelar som heltal eftersom `printf` med float ofta inte är påslaget på STM32.